

도미네이터를 활용한 지향성 협력 퍼저

모현민, 김윤희

한양대학교 컴퓨터소프트웨어학과
소프트웨어공학연구실



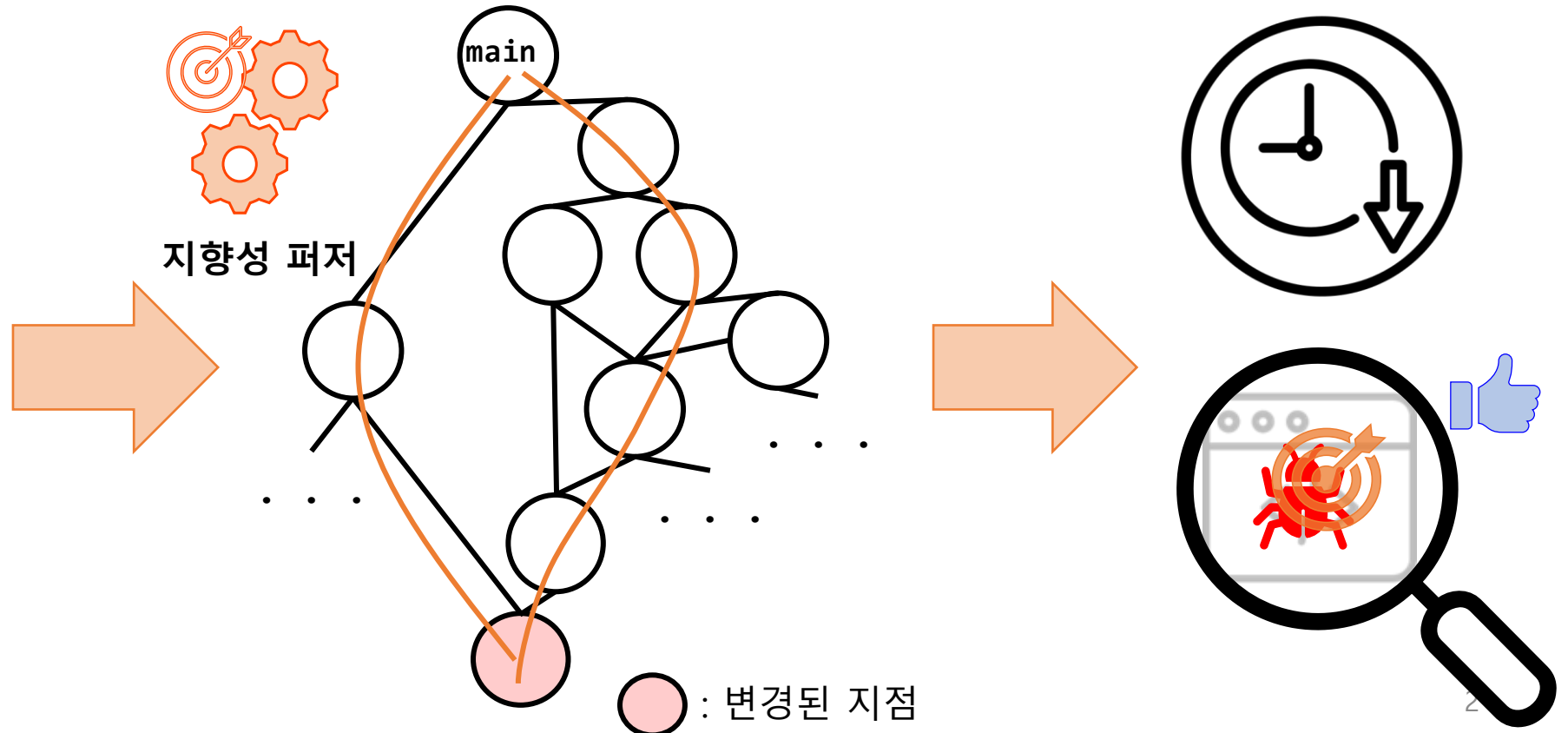
소프트웨어재난연구센터

지향성 퍼징

- 특정 목표 지점에 빠르게 도달하고 취약점을 재현할 수 있도록 탐색을 유도하여 입력을 자동 생성하는 퍼징 기법
 - 패치나 업데이트 등으로 변경된 지점에 빠르게 도달하여 오류 여부 검증하는 데 효과적임.



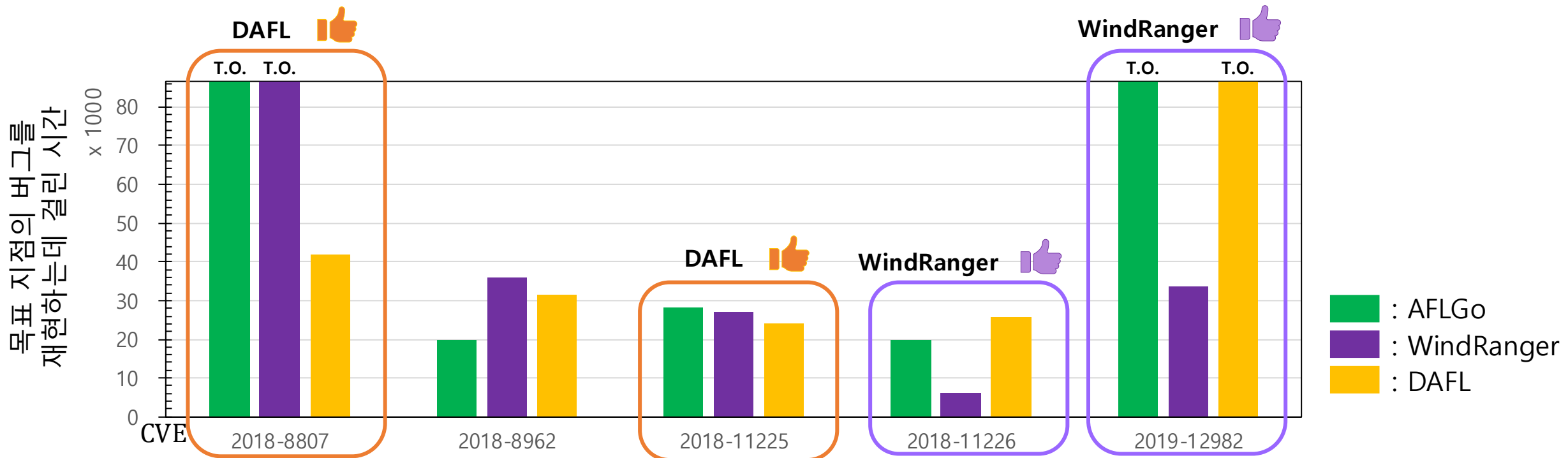
패치나 업데이트된 프로그램



단일 지향성 퍼저의 한계

- 같은 프로그램에서도 목표 지점마다 각 지향성 퍼저의 결과가 달라서 어떤 지향성 퍼저를 사용하는 것이 좋을 지 알기 어려움.
 - swftophp-2018-8807 과 2018-11225 에서는 **DAFL** 이 가장 성능이 좋음.
 - swftophp-2018-11226 과 2019-12982 에서는 **WindRanger** 가 가장 성능이 좋음.

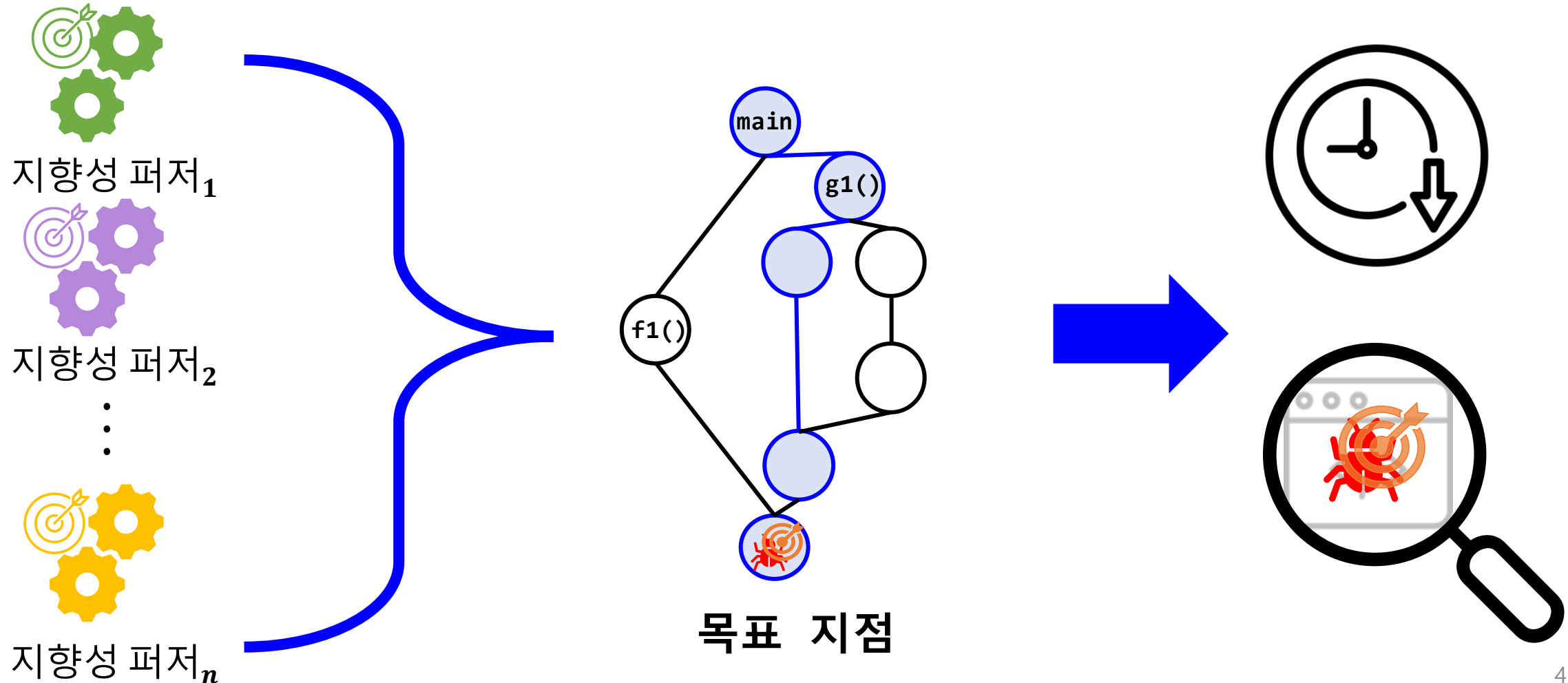
Ex) swftophp 프로젝트의 목표 지점에 대한 각 퍼저의 TTE 결과¹



❖ ¹각 지향성 퍼저를 24시간 씩 10회 실행하여 평균 TTE 측정한 결과

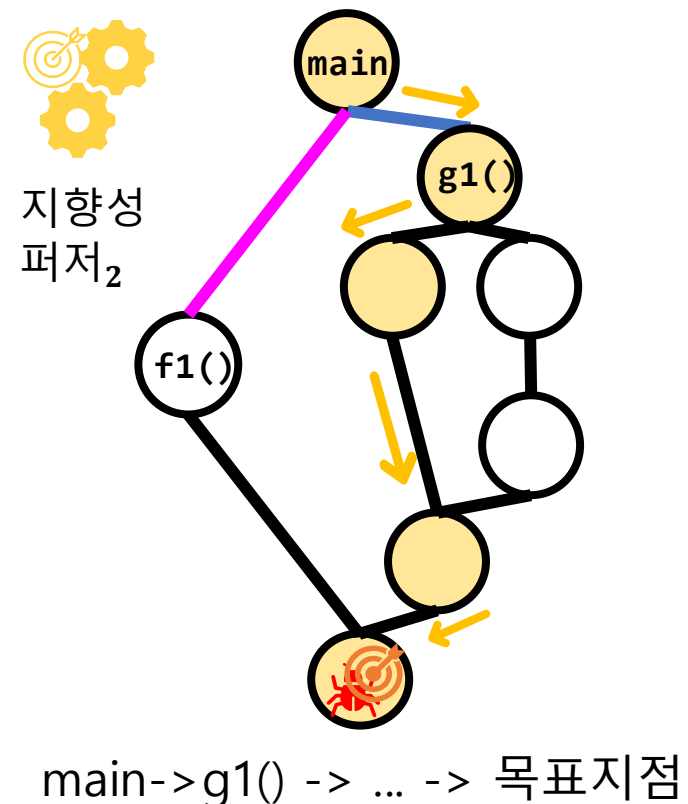
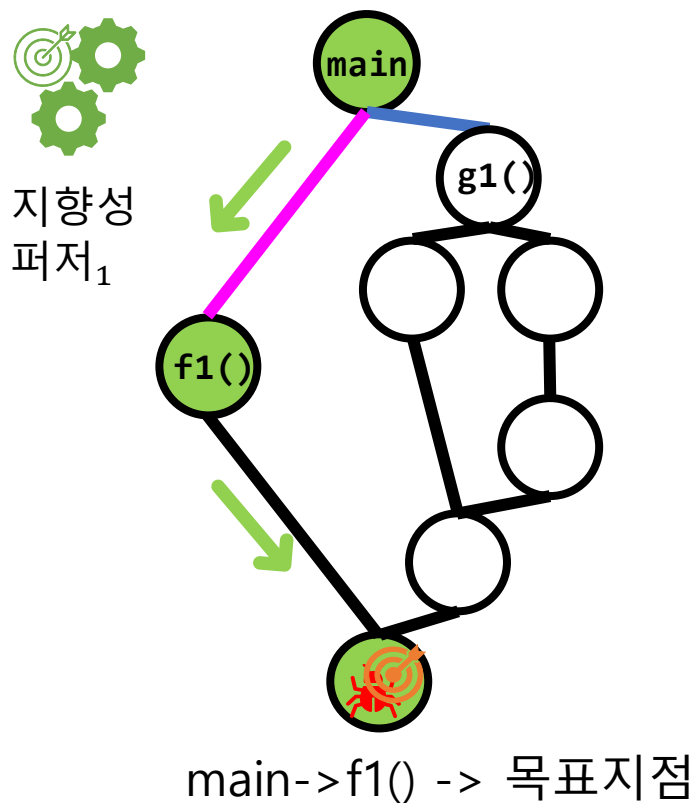
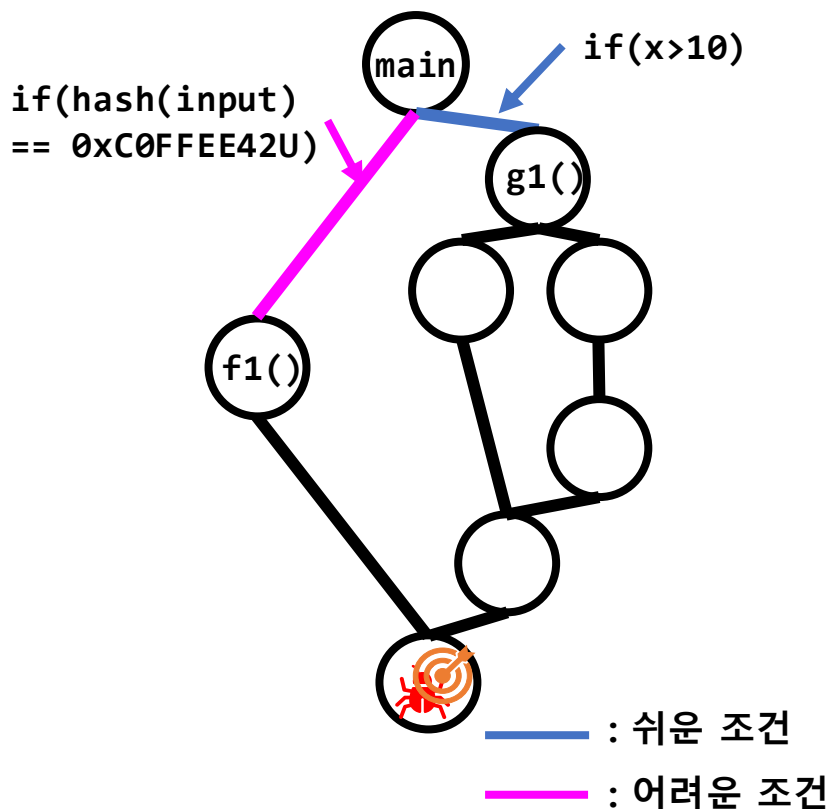
지향성 협력 퍼저

- 서로 다른 지향성 퍼저들의 전략 강점을 조합하여 목표 지점에 빠르게 도달하고 취약점을 빠르게 재현하기 위해 협력하는 퍼징 기법



협력을 고려한 성능 피드백 부재

- 각 지향성 퍼저마다의 거리 측정 방식이 달라서 어떤 지향성 퍼저가 목표 지점에 더 가깝게 도달했는지 판단하기 어려움.

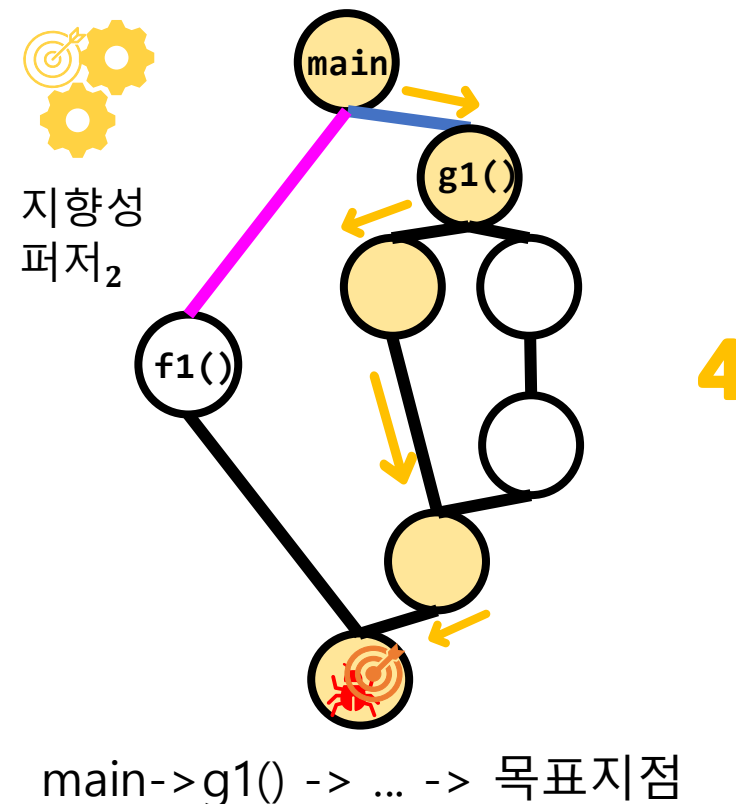
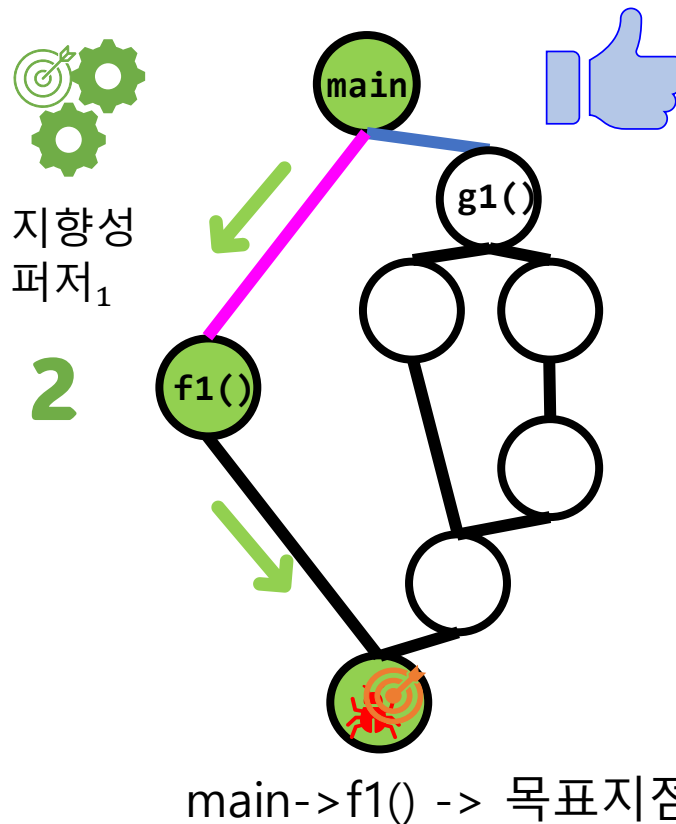
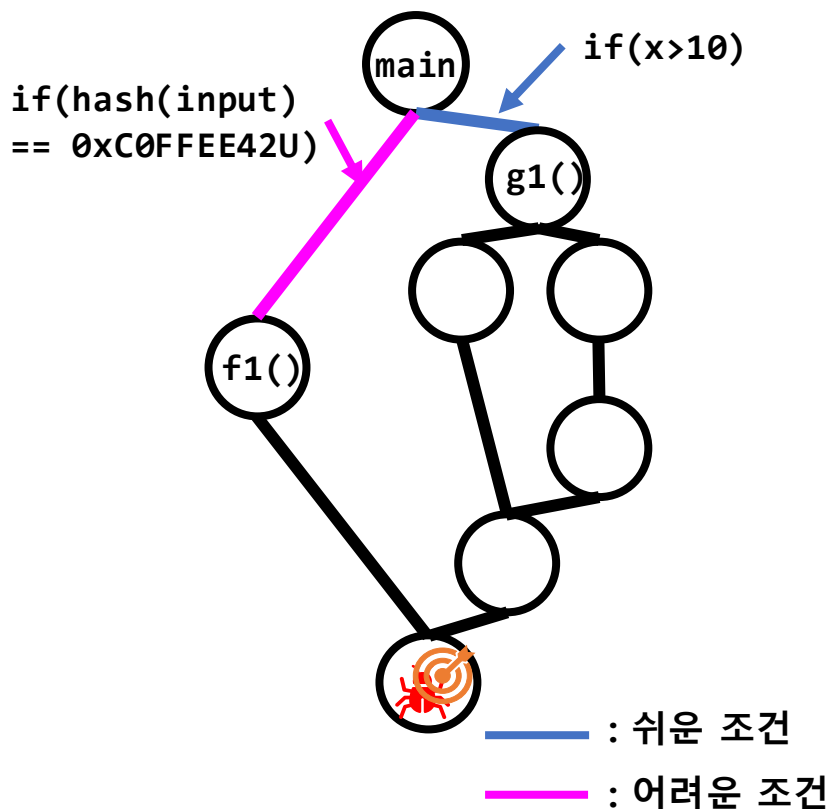


협력을 고려한 성능 피드백 부재

- 각 지향성 퍼저마다의 거리 측정 방식이 달라서 어떤 지향성 퍼저가 목표 지점에 더 가깝게 도달했는지 판단하기 어려움.

❖ 거리(엣지 수)로 평가

➢ 목표 지점까지 통과한 엣지 수가 적은 쪽이 좋은 퍼저

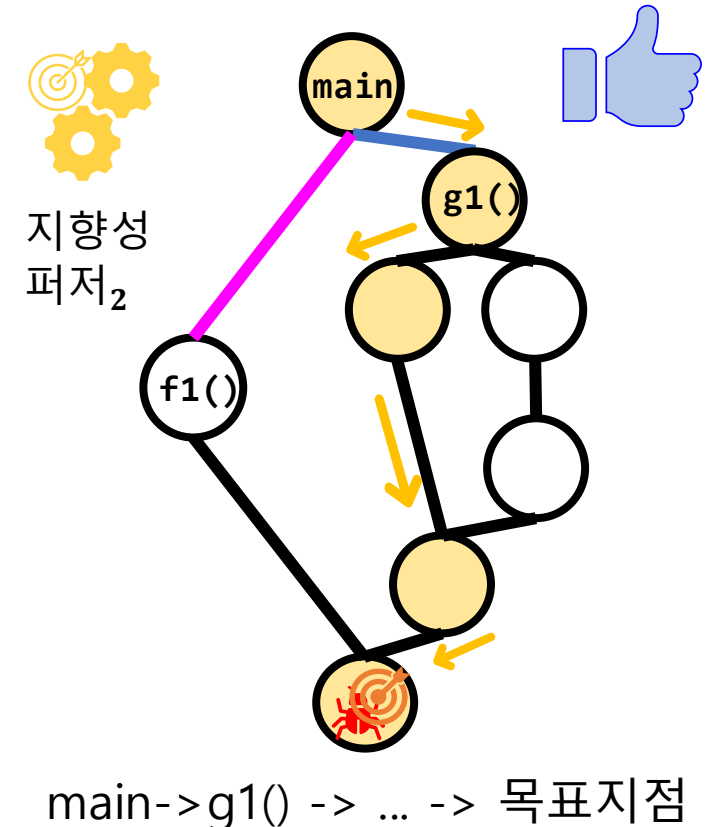
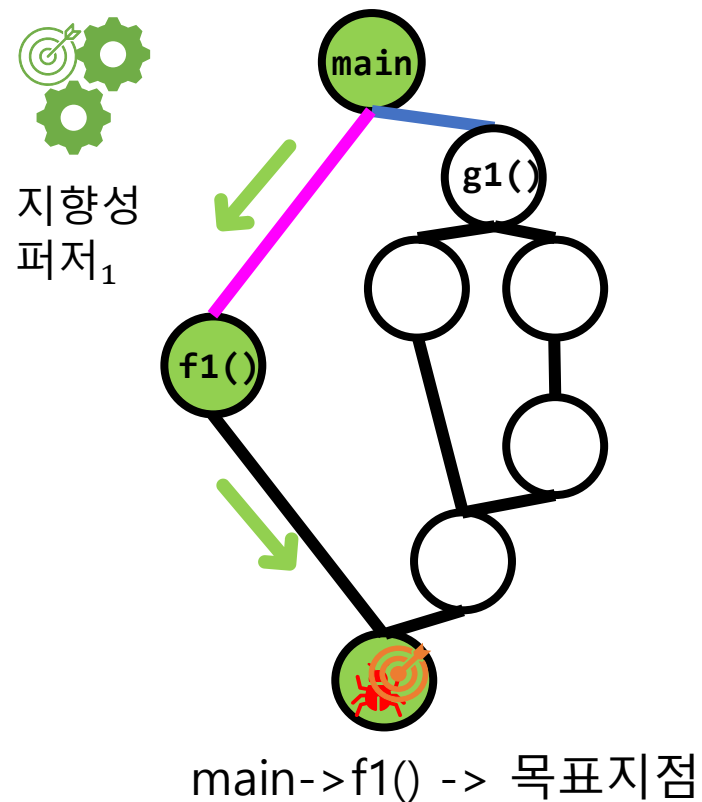
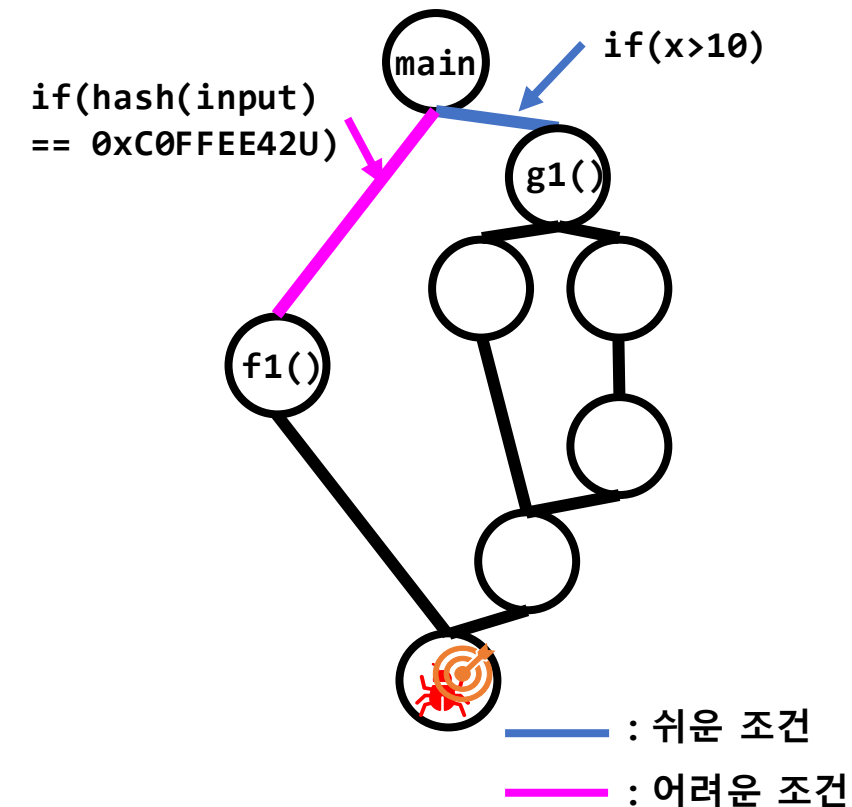


협력을 고려한 성능 피드백 부재

- 각 지향성 퍼저마다의 거리 측정 방식이 달라서 어떤 지향성 퍼저가 목표 지점에 더 가깝게 도달했는지 판단하기 어려움.

❖ 분기 조건으로 평가

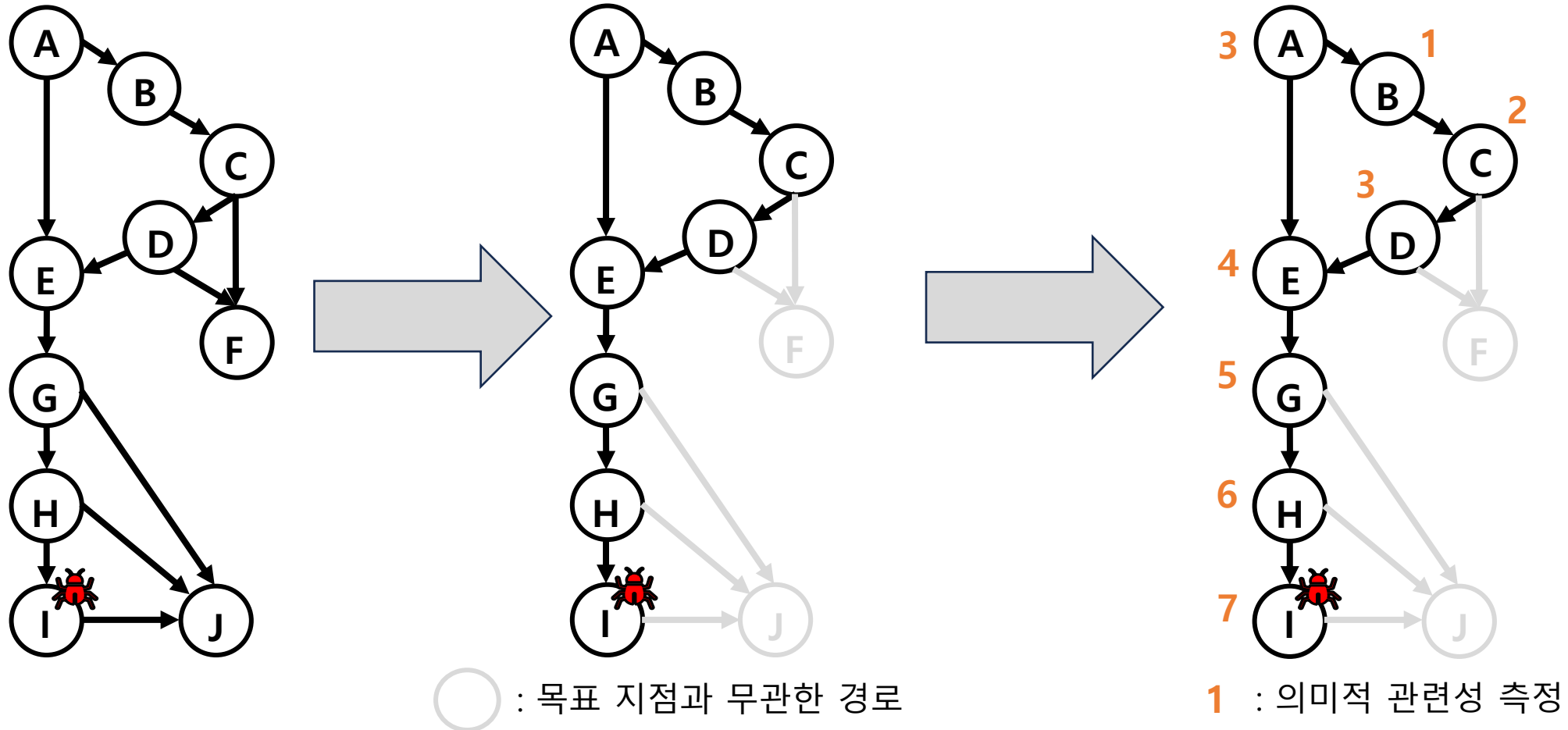
➢ 목표 지점까지 상대적으로 쉬운 분기를 통과한 쪽이 좋은 퍼저



공통된 평가 기준

- 의미적 관련성 점수를 활용한 공통된 평가

- 목표 지점과 무관한 경로 측정을 제한하여 목표 지점과 관련된 코드 부분만 측정함.
- 목표 지점 거리와 데이터 연관성을 결합한 의미적 관련성 측정을 통해 더 정밀하게 평가함.



의미적 관련성 점수 편향 문제

- 목표 지점에 더 근접한 실행 경로보다 목표 지점과 연관된 점수를 더 많이 누적한 실행 경로가 더 우수하게 평가됨.
 - 의미적 관련성 점수의 누적 특성으로 인해 목표 지점과 연관된 노드를 많이 포함한 긴 실행 경로가 더 높은 점수를 얻기 때문

❖ 지향성 퍼저₁ : **A** → **B** → **D** → **F** → **G** → **I** → **K**



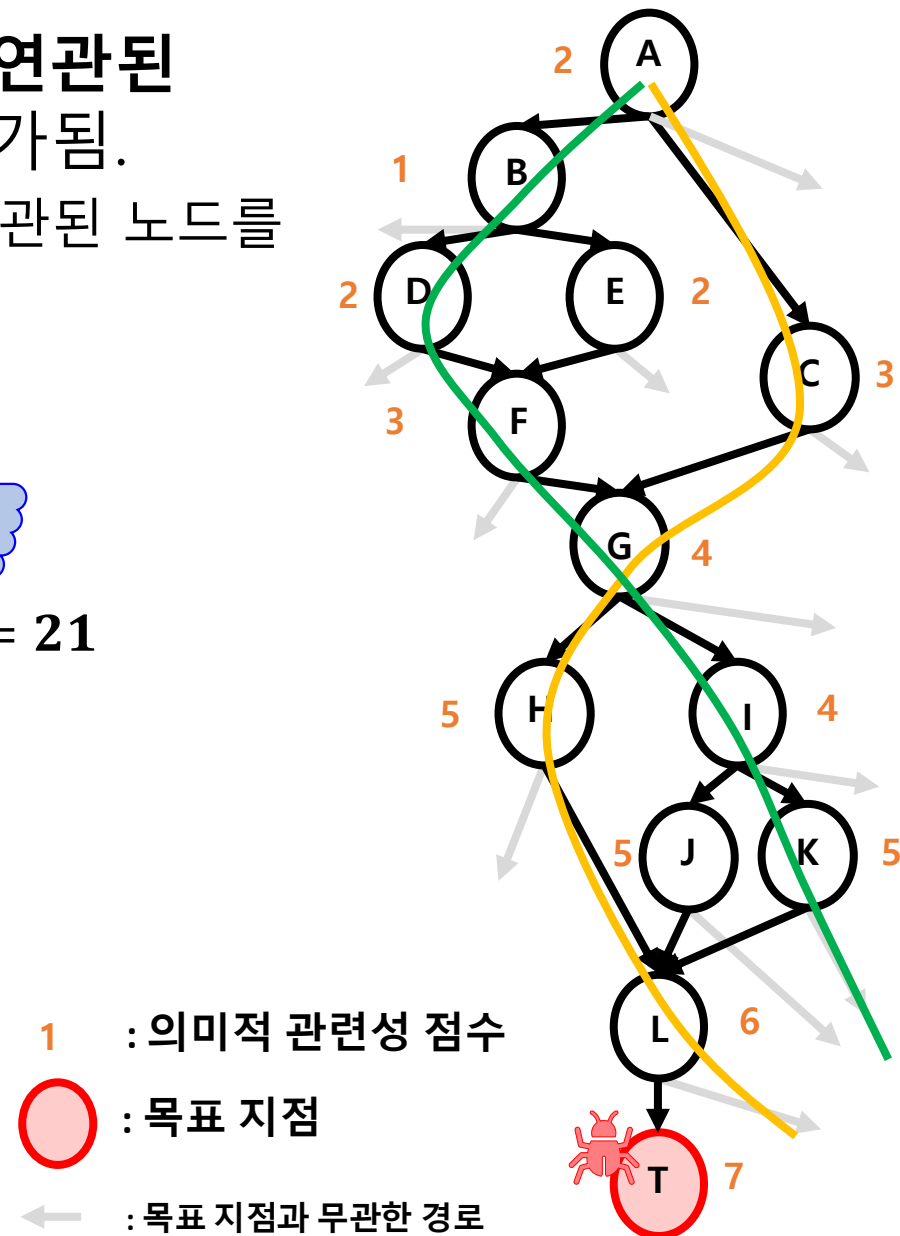
실행 경로의 의미적 관련성 점수 = $2 + 1 + 2 + 3 + 4 + 4 + 5 = 21$

목표 지점까지 남은 경로 : **K** → **L** → **T**

❖ 지향성 퍼저₂ : **A** → **C** → **G** → **H** → **L**

실행 경로의 의미적 관련성 점수 = $2 + 3 + 4 + 5 + 6 = 20$

목표 지점까지 남은 경로 : **L** → **T**



공통된 평가 기준 보완

- 도미네이터 노드 통과 수를 기준으로 목표 지점 도달 정도를 평가하여 실행 경로 길이에 따른 점수 편향 문제를 보완
 - 도미네이터 노드 : 목표 지점 도달을 위해 반드시 통과해야 하는 노드
 - 즉, 더 많은 도미네이터 노드를 통과한 퍼저가 목표 지점에 더 가깝게 도달한 것으로 평가할 수 있음.

❖ 지향성 퍼저₁ : A → B → D → F → G → I → K

실행 경로의 의미적 관련성 점수 = $2 + 1 + 2 + 3 + 4 + 4 + 5 = 21$

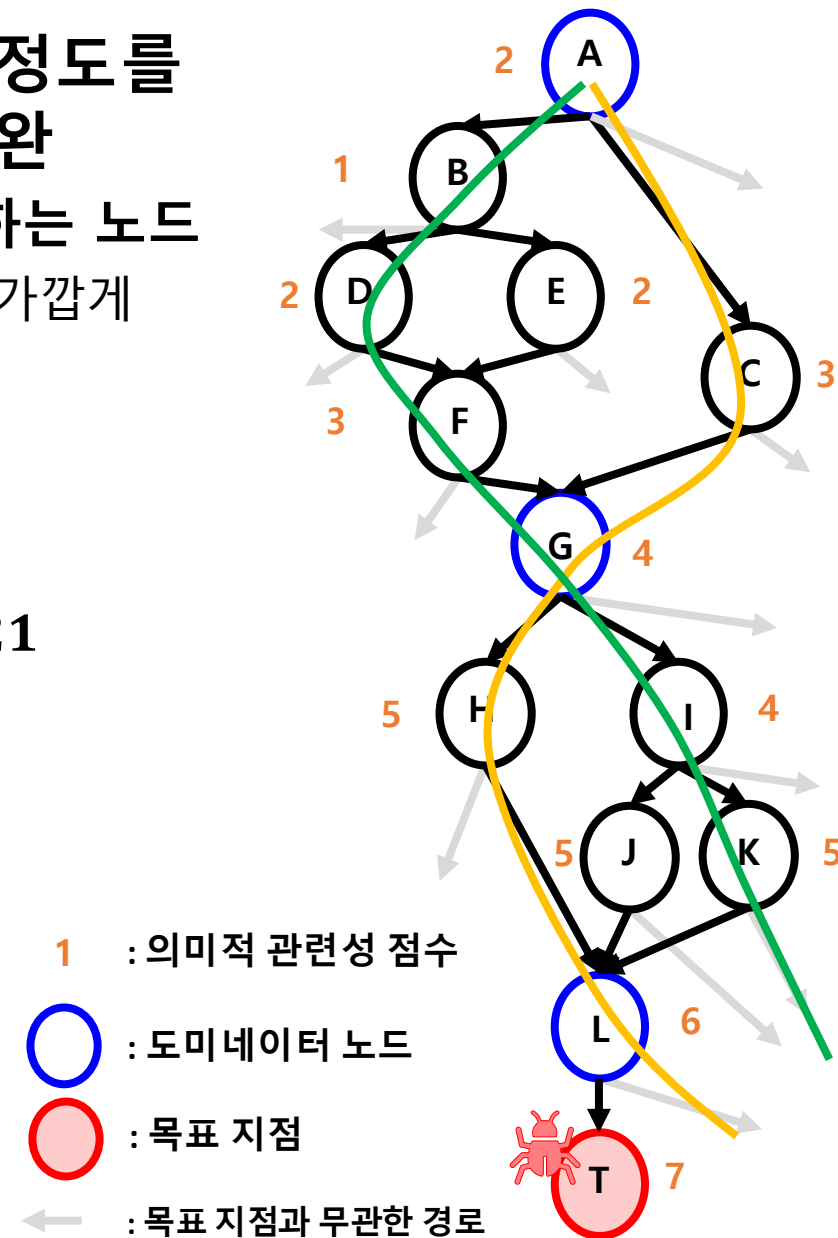
통과한 도미네이터 노드 수 = 2

❖ 지향성 퍼저₂ : A → C → G → H → L



실행 경로의 의미적 관련성 점수 = $2 + 3 + 4 + 5 + 6 = 20$

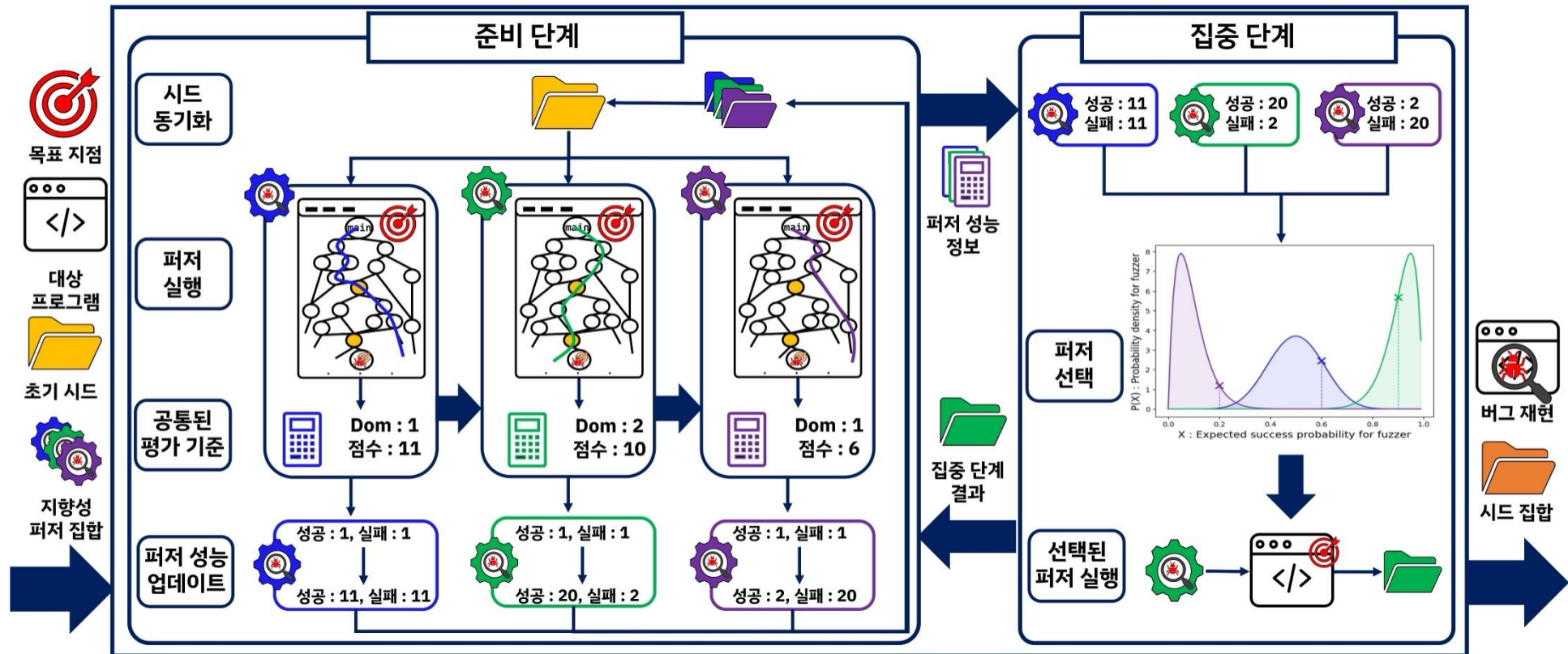
통과한 도미네이터 노드 수 = 3



DCFuzz

- DCFuzz(Directed Collaborative Fuzzer)

- **준비 단계:** 각 지향성 퍼저를 실행해서 **공통된 평가 기준**으로 성능 비교 및 수집하는 단계
 - 공통된 평가 기준 : 실행된 중간 목표 지점 수와 의미적 관련성 점수 측정
- **집중 단계:** 수집된 정보를 바탕으로 **탐슨 샘플링 기반 퍼저 선택**하여 목표 지점 도달 및 취약점 재현 효율을 극대화하는 단계



포스터 세션에서 뵙겠습니다.



소프트웨어재난연구센터



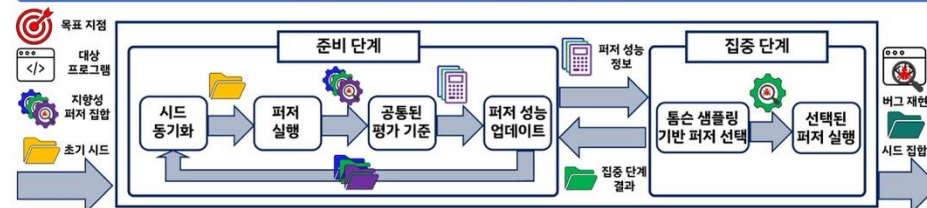
도메이너를 활용한 지향성 협력 퍼저

모현민, 김윤호
한양대학교 소프트웨어공학연구실

배경 및 동기

- 단일 지향성 퍼저의 한계
 - 같은 프로그램에서도 지향성 퍼저마다 결과가 다르기 때문에 최적의 지향성 퍼저를 선택하는 것이 어려움.
- 지향성 퍼저 성능 비교를 위한 공통된 기준 부재
 - 각 지향성 퍼저마다 거리 계산 방식(예: CFG, DFG, ...) 이 달라 퍼저 간 성능을 일관되게 비교하기 어려움.
- 전체 실험의 약 30.0%(= 총 210 라운드 중 65 라운드)에서 의미적 관련성 점수 편향 문제 발생
 - 의미적 관련성 점수 편향 문제: 목표 지점에 더 근접한 실행 경로보다 목표 지점과 연관된 점수를 더 많이 누락한 실행 경로가 더 우수하게 평가됨.

도메이너를 활용한 지향성 협력 퍼저



- 두 단계 기준 기반 공통된 평가
 - 각 퍼저가 생성한 시드를 거리 계산 방식과 달리 통과한 도메이너 노드 수를 기준으로 목표 지점 도달 진행 정도를 일관되게 비교한 후 의미적 관련성 점수로 취약점 재현 가능성 비교

❖ 1 단계 비교 기준: 통과한 도메이너 노드 수

- 더 많은 도메이너 노드를 통과한 퍼저가 목표 지점에 더 가깝게 도달한 것으로 판단할 수 있음.
- 도메이너 노드: 목표 지점 도달을 위해 반드시 통과해야 하는 노드

$$C_1 : \text{Dom}(s, t) = |C(s) \cap D_t|$$

- $C(s)$: 시드 s 의 실행 경로에서 방문한 노드 집합
- D_t : 목표 지점 t 까지 도달하는 데 반드시 통과해야 하는 도메이너 노드 집합

❖ 2 단계 비교 기준: 의미적 관련성 점수

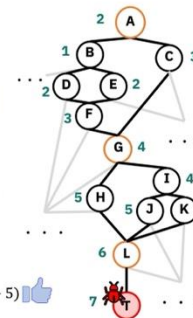
- 실행 경로에서 목표 지점과 의미적으로 관련된 노드를 많이 지난 퍼저가 목표 지점 취약점 재현 가능성이 높다고 판단
- 의미적 관련성 점수: 실행 경로에서 목표 지점과 의미적으로 관련된 노드의 점수를 누적한 값

$$C_2 : \text{Score}_{G,t}(s) = \sum_{c \in C_t} (L - |c - t| + 1)$$

- L : 목표 지점으로 부터 가장 먼 지점까지의 거리
- C_t : 정역-사용 그래프 상에서 목표 지점 t 의 변수 등과 관련이 있는 노드 집합

- 도메이너를 추가한 이유: 의미적 관련성 점수 편향 문제를 보완하여 목표 지점 도달과 취약점 재현 가능성을 정확히 평가하여 최적의 퍼저 선택하기 위함. (약 84.6% (= 65 라운드 중 55 라운드) 최적의 퍼저 선택함.)

- 지향성 퍼저₁: $A \rightarrow B \rightarrow D \rightarrow F \rightarrow I \rightarrow K \rightarrow \dots$ Dom = 2 Score = 21 (= 2 + 1 + 2 + 3 + 4 + 4 + 5)
- 지향성 퍼저₂: $A \rightarrow C \rightarrow G \rightarrow H \rightarrow L \rightarrow \dots$ Dom = 3 Score = 20 (= 2 + 3 + 4 + 5 + 6)
- 지향성 퍼저₃: $A \rightarrow B \rightarrow E \rightarrow F \rightarrow G \rightarrow \dots$ Dom = 2 Score = 12 (= 2 + 1 + 2 + 3 + 4)



실험 환경

- 벤치마크 (총 21개): 기존 지향성 퍼저 연구에서 사용된 CVE 로 선정함.
- 비교 대상 퍼저: AFLGo, Windranger, DAFL
- 평가 지표: 12시간씩 10회 수행해서 평균 최초 목표 지점 도달 시간(TTR)과 평균 최초 목표 지점 취약점 재현 시간(TTE)로 측정함. (단위: 초)

연구 질문 1 - 최초 목표 지점 도달 시간 비교

- DCFuzz는 AFLGo, Windranger, DAFL에 비해 15개, 15개, 10개 목표 지점에서 더 빠르게 목표 지점에 도달하고 전체 평균 순위 1.5위

Prog	CVE	A	W	D	DC
cex _int	2016-4489	581.4	307.9	758.2	1119.5
	2016-4490	220.4	181.6	46.8	39.9
	2016-4491	2724.7	4656.1	5020.8	2430.0
	2016-4492	556.9	4020.5	449.4	1091.3
	2016-6131	4678.1	4883.8	935.8	1334.0
nm _to _php	2017-14940	43200.0	43200.0	43200.0	43200.0
	2016-9827	7816.6	2254.0	1520.2	376.5
	2016-9829	32545.8	13885.8	838.2	4235.9
	2019-9114	43200.0	43200.0	43200.0	43200.0
	2020-6628	17862.0	35582.5	15966.3	12088.8
평균 순위		2.7	2.6	1.8	1.5

A: AFLGo | W: Windranger | D: DAFL | DC: DCFuzz

연구 질문

- 연구 질문 1. DCFuzz는 단일 지향성 퍼저에 비해 빠르게 목표 지점에 도달하는가?
- 연구 질문 2. DCFuzz는 단일 지향성 퍼저에 비해 빠르게 목표 지점의 취약점을 재현하는가?

연구 질문 2 - 최초 취약점 재현 시간 비교

- DCFuzz는 AFLGo, Windranger, DAFL에 비해 15개, 16개, 10개 목표 지점에서 더 빠르게 목표 지점 취약점을 재현하고 전체 평균 순위 1.7위

Prog	CVE	A	W	D	DC
cex _int	2016-4489	214.1	161.8	217.7	497.3
	2016-4490	204.4	199.5	34.2	83.7
	2016-4491	140.8	138.4	31.7	26.4
	2016-4492	4225.7	3030.2	581.5	1199.0
	2016-6131	5.3	6.5	6.5	5.3
nm _to _php	2017-14940	0.0	0.0	0.0	0.0
	2016-9827	55.7	48.1	52.3	76.0
	2016-9829	272.8	156.6	118.0	109.4
	2019-9114	11597.3	10835.2	15904.7	16700.1
	2020-6628	43200.0	35176.3	14208.0	11871.0
평균 순위		3.0	2.7	1.9	1.7

A: AFLGo | W: Windranger | D: DAFL | DC: DCFuzz